Minecraft 与 LittleTiles
技术研究文档

08/03/2024

目录

Chapter 1 Anvil 格式

1.1 区块的种类

在《Minecraft》中，游戏的进本单位是方块，游戏将这些数据存储在称为区块的结构中，每个区块的大小为 16×16 方块，高度为 n 方块。区块被进一步地组织到地图块中，一个地图块包含 32×32 个区块

1.2 区块数据的存储

在 1.2 版本之后，《Minecraft》以 Anvil 格式存储，即 mca 文件。它存储的是地图块规范化后的数据，游戏运行时会加载需要的 mca 文件以解析出地图块。然后提取出需要的区块进行实时加载渲染。

1.2.1 Anvil 格式

Anvil 格式用于指导如何存储一个地图块数据。每个 mca 文件都是 Anvil 格式，并且只存储一个地图块，其文件名指出了该文件对应的地图块坐标（r.x.z.mca）。

在 Anvil 格式中，前 8KiB 被称为 8KiB 区，它用于存储区块的索引数据和区块最后修改时间，剩余部分是用于存储区块已压缩数据的多个扇区。每个扇区的大小为 4096 字节，一个区块可以占用多个扇区。

因此 mca 文件大小一定为 4096 字节的倍数，Minecraft 不接受非 4096 字节的倍数的 mca 文件。

1.2.2 8KiB 区

8KiB 区分为两个 4KiB 区，其中前者提供区块的索引相关数据，后者提供区块最后修改的时间戳。

两个 4KiB 区都是以 4 字节为单位进行分割的，其分别存储 $[x, z]$ 区块的偏移量和 $[x, z]$ 区块的最后修改时间。

注意 x, z 从 0 开始遍历，且先遍历 x，再遍历 z：

$$[0,0], [1,0], [2,0], \dots, [31,0], [31,1], \dots, [31,31]$$

前 4KiB 区每组的前 3 字节为偏移量，以大端序存储，单位为 4096 字节，后 1 字节为该区块所用的扇区数量：

Byte	1	2	3	4
描述	偏移量			扇区数量

要计算 $[x,z]$ 区块在前后 4KiB 区中的具体字节位置，可通过如下公式得到：

$$4 \times \big((x \ mod \ 32) + (z \ mod \ 32) \times 32\big)$$

在编程中，如 Java/C/C++ 可能会出现负数，所以需要使用 AND 运算符而不是取模来进行计算：

$$4 \ * \ ((x \ \& \ 31) \ + \ (z \ \& \ 31) \ * \ 32)$$

例如，区块 [3,12] 计算后为 1548 字节，即该区块的偏移量在前 4KiB 区的第 1548 字节的位置。最有以大端序读取偏移量，并将结果乘以 4096 即可得到该区块在 mca 文件中的字节位置，然后根据其扇区数量读取对应的扇区。

到这并没有真正获取到区块的数据，只是获取了该区块的压缩数据。

最后修改时间同理，在后 4KiB 的第 1548 字节处。

1.2.3 区块数据

在上一节中我们获取了指定区块在文件中对应的扇区位置，在每个区块第一个扇区的起始位置，有 5 个字节用于记录该区块压缩数据的信息，前 4 个字节存储有效数据的长度（单位为字节），后 1 个字节存储压缩类型标识符：

Byte	1	2	3	4	5
描述	有效数据长度				压缩类型

目前压缩类型定义了三种方案：

标识符（十进制）	方案
1	GZip （RFC1952）（未使用）
2	Zlib （RFC1950）
3 (1.15.1 之前的版本)	未压缩 （未使用）

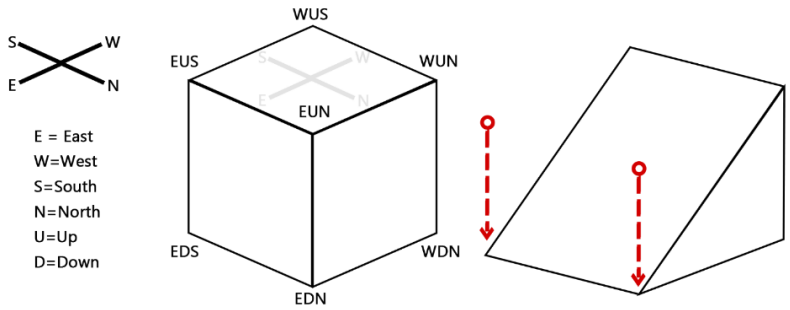
当获取到有效数据长度时，可以提取其对应的压缩数据，并根据压缩类型对这段数据进行解压，即可得到 NBT 的二进制格式。

注意这里指的是原始二进制格式，而不是解析后的 SNBT 格式或解析后的树状 NBT 结构。

Chapter 2　LittleTiles 的存储

2.1　Little tiles 的基本单位

LittleTiles 的 Tile 是以《Minecraft》中的一个方块为单位合并存储的。并将这些数据存储到区块文件中。每个 Tile 实际上是一个立体矩形，即使它是一个斜面；它由八个顶点组成：



上图可以看到这八个顶点（除了隐藏在后面的 WDS），它们的命名依照四个方向命名，在名字中间添加 U/D 以表示它是上面的顶点还是下面的顶点。

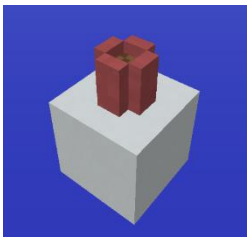
例如 EUS 就代表东南方向，上方，如图所见。

有时候你会选择去做一个斜面，打破《Minecraft》的限制。在制造斜面的时候，实际上只是把这八个顶点中的一些或全部进行了三个方向的移动（x,y,z）。

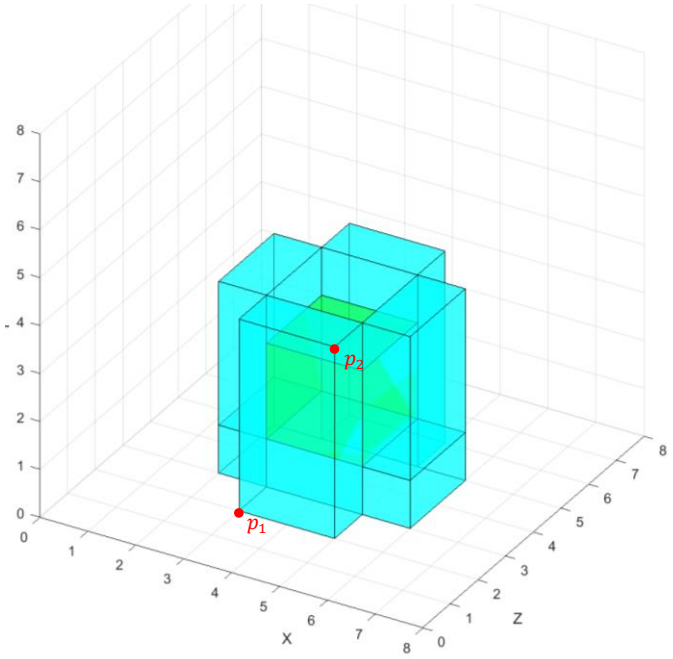
例如上图右侧的图形，只是把 EUS 和 EUN 沿着 y 轴向下移动，直到与 EDS 和 EDN 重合。具体的偏移数值，与 Tile 和方块的细分级别（grid）都有关联，后续介绍。

2.2　Little tiles 的块

上面我们了解到，Tile 存储于方块之中，并以此为单位存储在区块之中。那么在一个方块中，体素的存储我们有一个花盆示例，它周围包裹着粉红的陶瓦，中间是泥土。



它的精度为 8，具体如右图所示：



在左侧图中，青色部分就是粉红色陶瓦，而中间绿色的部分就是泥土。

我们可以清晰的看出这个三维图像在 8x8x8（精度）的空间内组合了多个 Tile，最终形成我们在游戏里的画面。

每个体素都有两个坐标，它定义了立体矩形在这个三维空间中的两个顶点。

例如最外面的那个 Tile，它在该块中的坐标就是:

$$p_1(3,0,2), p_2(5,4,3)$$

具体底层存储主要数据如下：

```
Tiles info (BLOCK):
        block coord: 1 5 2                  // 方块坐标
        grid: 8                             // 精度（细分等级）
        id: minecraft:littletilestileentity
        boxes count: 2

      Tile info:
            block id: minecraft:dirt        // Tile 的方块类型
            boxes:  [3, 1, 3, 5, 3, 5]      // 定义了立体矩形两顶点坐标
                                            // [x1, y1, z1, x2, y2, z2]

      Tile info:
            block id: minecraft:stained_hardened_clay:6
            boxes:  [2, 0, 3, 6, 1, 5]
                    [2, 1, 3, 3, 4, 5]
                    [5, 1, 3, 6, 4, 5]
                    [3, 0, 5, 5, 4, 6]      // 示例中提到的那个 Tile
                    [3, 0, 2, 5, 4, 3]
```

2.3 体素的具体表示

2.3.1 角的偏移

之前我们说 Tile 有八个顶点，这八个顶点在方块中可以通过立体矩形的两个坐标进行确定。所以表示八个顶点的位置，只需知道它在方块中的两个坐标即可确定。

所以你会在 SNBT 标签中看到一个数组，它的前 6 位表示体素在块中的坐标。

$$bBox: [I; x_1, y_1, z_1, x_2, y_2, z_2, \dots]$$

$$\begin{cases} p_1 = (x_1, y_1, z_1) \\ p_2 = (x_2, y_2, z_2) \end{cases}$$

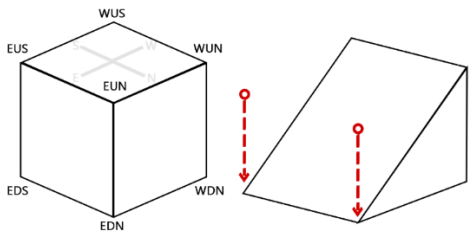
而后面有时候也会跟一堆数字，这些数字就是该 Tile 八个顶点的偏移数据。首先坐标后面紧跟着的第一个数字，它表示了哪些顶点进行了偏移，哪些没有。后面的数据则是按顺序列出的具体偏移量。

符号位永远为负(1)	空	Flipped						角																							
								WDS			WDN			WUS			WUN			EDS			EDN			EUS			EUN		
		E	W	S	N	U	D	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X
1	0																														
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

上图从高位到低位有序列出了该 32 位有符号整数的每一个位的作用。符号位永远位 1，它代表负号。是 0 时，那它就成了正数（当然 Lt 不希望出现这样的情况）。

我们先从右往左看，每三个位代表一个顶点，它定义了一个顶点的 x,y,z 这三个轴。如果哪个轴移动了，它就会标识为 1，没有则是 0。

我们回到之前的例子，假设它的精度是 2。此时我对 EUS 和 EUN 都沿着 y 轴向下偏移了 2，那么它应该是右边的图形：



这张图的位设定是这样的：

符号位永远为负(1)	空	Flipped						角																							
								WDS			WDN			WUS			WUN			EDS			EDN			EUS			EUN		
		E	W	S	N	U	D	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

我们可以看到 EUS 和 EUN 的 Y 都被设置成了 1，这说明这两个角都沿 y 轴进行了偏移。

这段二进制转换为十进制就变成了-2147483630。这也是为什么我们在导出 Lt 蓝图的时候，那串字符中可能出现许多很大的负数。

$$bBox: [I; 0,0,0,1,1,1, -2147483630, \dots]$$

既然我们知道了如何设定哪些角进行了偏移，那么后面的数字就代表了被偏移的角具体偏移了多少。它按照顺序排列，让我们先忽略掉哪些角，只看下一行的那些 xyz。其实这些 xyz 都表示是否沿着这个轴进行了偏移，所以我们也忽略掉这些 xyz，只看是否有轴被偏移。

所以我们知道，最多需要 24 个值来表示这 24 个偏移量/是否偏移（8×3）。但是每次把没有偏移的角都在后面进行存储，会非常浪费空间。所以我们只存储偏移过的角。

在例子中我们偏移了 EUN 的 Y 和 EUS 的 Y，所以我们会在后面需要两个值来表示。它的顺序就是从右往左依次写入偏移值的。

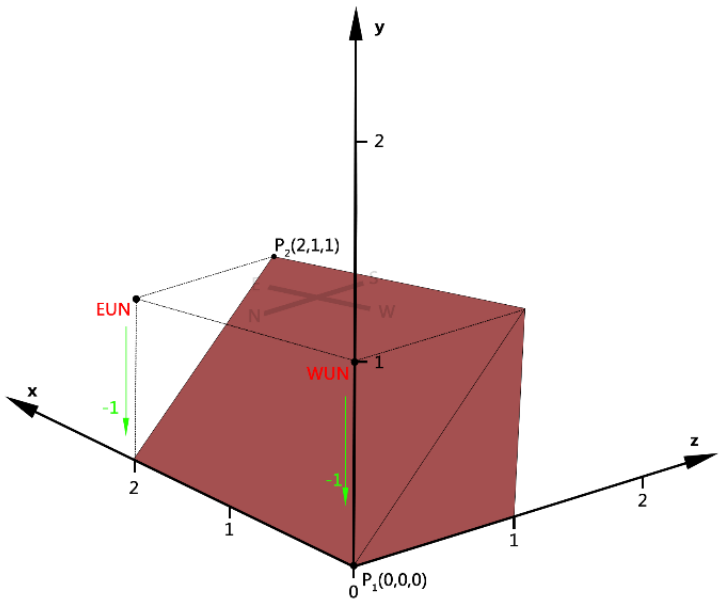
所以先写入 EUN 的 Y，然后才是 EUS 的 Y。读取的时候也是根据 Tile 坐标后面的那个很大的负数，查看哪些角有偏移。然后读取后面的数据，按照表从右往左的顺序读入，去定义后面的值都是哪些轴的偏移量。没有偏移的不进行读取，直接设为 0。

但是，这些角偏移量是以 32bit 为单位存储的，但是偏移量只有 16bit，所以在读取的时候，我们需要将 32bit 的有符号数拆成两份 16bit 的有符号数，它们分别都代表了一个偏移量。

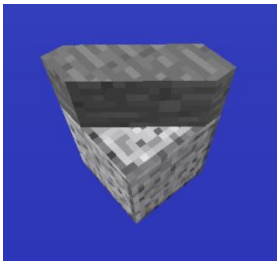
所以数组中的第 8 及以后的数，它实际上每个数都代表了两个偏移量。

另一个更直观的例子是：

```
Tiles info (BLOCK):
     block coord: 15 4 10
     grid: 2
     id: minecraft:littletilestileentity
     boxes count: 1
     Tile info:
          block id: minecraft:stained_hardened_clay:6
          boxes:  [0, 0, 0, 2, 1, 1, -2147475454, -1]
          Angle change state (from -2147475454):
          |FLIPPED|WDS|WDN|WUS|WUN|EDS|EDN|EUS|EUN|
          | EWSNUD|zyx|zyx|zyx|zyx|zyx|zyx|zyx|zyx|
          | 000000|000|000|000|010|000|000|000|010|
          Angle offset (from -1):
          WUN:  y_offset: -1
          EUN:  y_offset: -1
tips:
  P_1(0,0,0) 来自 [0,0,0,2,1,1,...]
  P_2(2,1,1) 来自 [0,0,0,2,1,1,...]
  -1 的二进制表示为 11111111,11111111,11111111,11111111 (32bit)
  WUN y_offset 来自前16位，即 11111111,11111111
  EUN y_offset 来自后16位，即 11111111,11111111
  grid 代表该方块内的小方块精度是 2
```

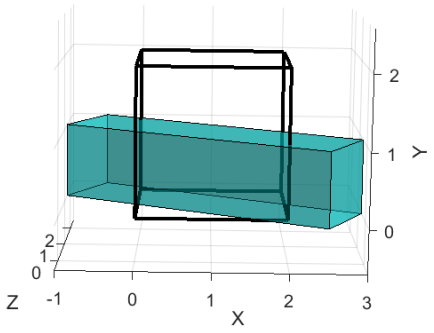


对于一些其它斜面，它的偏移量可能会超出方块的边界，例如右侧的小方块，它的精度为 2，各顶点的偏移量为：

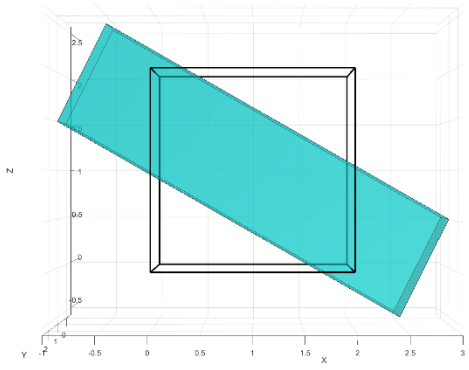


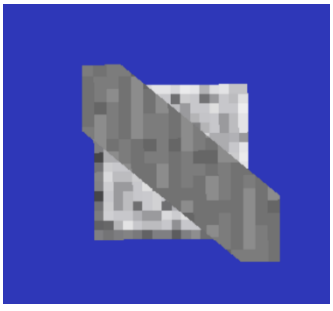
WDS:	x_offset: -2	z_offset: 2
WDN:	x_offset: 3	z_offset: -2
WUS:	x_offset: -2	z_offset: 2
WUN:	x_offset: 3	z_offset: -2
EDS:	x_offset: -3	z_offset: 2
EDN:	x_offset: 2	z_offset: -2
EUS:	x_offset: -3	z_offset: 2
EUN:	x_offset: 2	z_offset: -2

我们可以发现，它的顶点均超出了方块边界，它的实际效果是这样：



我们在游戏中看到的只是根据方块边界裁剪后的立体图形：



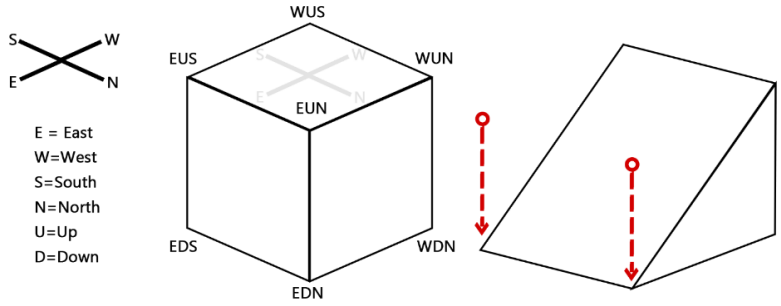


2.3.2 Flipped 位的作用

Chapter 3 Study on the Storage Structure of LittleTiles

3.1 Basic Units of LittleTiles

In the LittleTiles mod, a tile is stored as a unit of a Minecraft block and saved in NBT format within the chunk files. A tile is essentially a three-dimensional rectangle, composed of eight vertices:



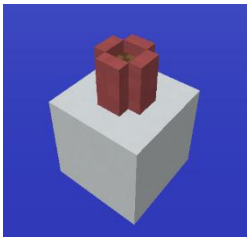
In the diagram, you can see these eight vertices (except for the ones hidden behind WDS). They are named according to the cardinal directions, with "U" or "D" added to indicate whether the vertex is on the top or bottom.

For example, "EUS" represents the southeast direction on the upper side, as shown in the diagram.

Sometimes, you might choose to create a slope, breaking the constraints of Minecraft. When creating a slope, you essentially move some or all of these eight vertices.

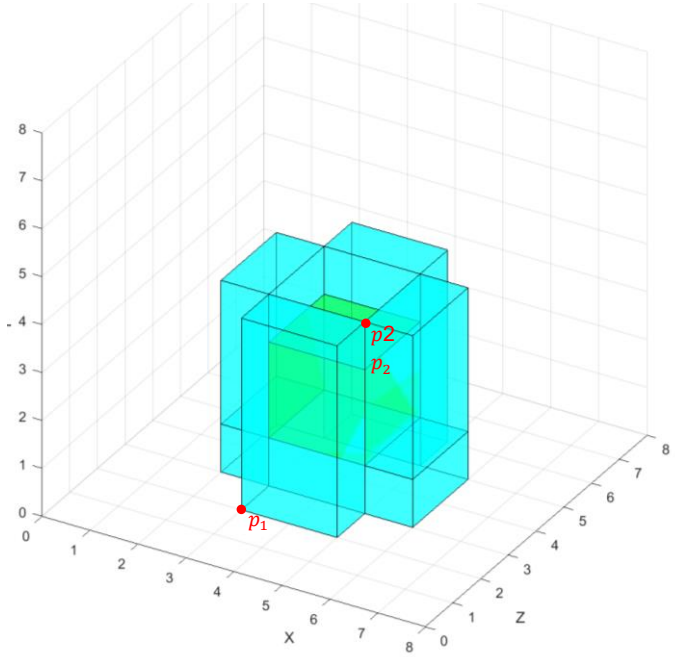
For example, in the shape on the right side of the diagram, the EUS and EUN vertices are simply moved downward along the y-axis until they align with the EDS and EDN corners. The specific offset values are related to voxel and block sizes, which will be discussed further later.

3.2 LittleTiles' Tile



As we learned earlier, tiles are stored within blocks as units and, in turn, stored within chunks. For example, in a block, the storage of tiles can be illustrated with a flower pot example. This example shows a flower pot surrounded by pink ceramic tiles, with soil in the center.

The precision of this storage is 8, as illustrated in the diagram below:



In the left-side diagram, the cyan areas represent the pink ceramic tiles, while the green area in the center represents the soil.

We can clearly see that this three-dimensional image combines multiple tiles within an 8x8x8 (precision) space, ultimately forming the visual we see in the game.

Each tile has two coordinates that define two vertices of the rectangular prism in this three-dimensional space.

For example, the outermost tile has the following coordinates within the block:

$$p_1(3,0,2), p_2(5,4,3)$$

The specific base layer storage of the main data is as follows:

```
Tiles info (BLOCK):
        block coord: 1 5 2
        grid: 8
        id: minecraft:littletilestileentity
        boxes count: 2

      Tile info:
            block id: minecraft:dirt
            boxes:  [3, 1, 3, 5, 3, 5]     // Defines the coordinates of the two points
                                           // of the rectangular prism  [x1, y1, z1, x2, y2, z2]

      Tile info:
            block id: minecraft:stained_hardened_clay:6
            boxes:  [2, 0, 3, 6, 1, 5]
                    [2, 1, 3, 3, 4, 5]
                    [5, 1, 3, 6, 4, 5]
                    [3, 0, 5, 5, 4, 6]    // The tile described in the example
                    [3, 0, 2, 5, 4, 3]
```

3.3 Detailed Explanation of a Tile

3.3.1 Vertex Offset

Previously, we mentioned that a tile has eight vertices. These eight vertices in a block can be determined by two coordinates of the rectangular prism. Thus, to represent the positions of the eight vertices, you only need to know the two coordinates within the block.

Therefore, in the SNBT tags, you will see an array where the first 6 values represent the tile's coordinates within the block (with the exact values determined by the subdivision level (grid) of the block).

$$bBox: [I; x_1, y_1, z_1, x_2, y_2, z_2, \dots]$$

$$\begin{cases} p_1 = (x_1, y_1, z_1) \\ p_2 = (x_2, y_2, z_2) \end{cases}$$

Sometimes, there will also be a series of numbers following these coordinates. These numbers represent the offset data for the eight vertices of the voxel.

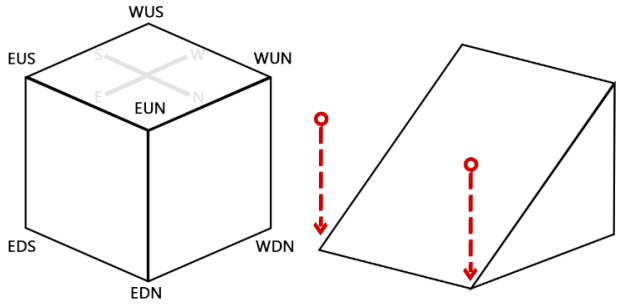
The first number after the coordinates indicates which vertices have been offset and which have not. The subsequent numbers provide the specific offset values in sequence.

Sign bit always negative (1)		Flipped						Vertex																					
								WDS			WDN			WUS			WUN			EDS			EDN			EUS			EUN		
		E	W	S	N	U	D	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X
1	0																										1			1	
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

The diagram lists the function of each bit in this 32-bit signed integer from high to low order. The sign bit is always 1, representing the negative sign. If it were 0, it would indicate a positive number (though LittleTiles does not expect such a situation).

Let's start from the right and move to the left. Every group of three bits represents one vertex, defining which of the three directional axes (x, y, z) the vertex can be offset along. A bit of 1 indicates movement along that axis, while 0 indicates no movement.

Returning to the previous example, assuming the precision is 1, if I offset both the EUS and EUN vertices downward along the y-axis by 1, the result should be the shape shown in the image below:



The bit settings for this image are as follows:

Sign bit always negative (1)		Flipped						Vertex																						
								WDS			WDN			WUS			WUN			EDS			EDN			EUS			EUN		
		E	W	S	N	U	D	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	Y	X	Z	**Y**	X	Z	**Y**	X
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	**1**	0	0	**1**	0
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

We can see that the Y values for EUS and EUN are both set to 1, indicating that both vertices have been offset along the y-axis.

This binary sequence converts to a decimal value of -2,147,483,630. This is also why we see many large negative numbers in the string of characters when exporting LittleTiles blueprints.

$$bBox: [I; 0,0,0,1,1,1, -2147483630, \dots]$$

Since we know how to determine which vertices have been offset, the subsequent numbers represent the specific amount of offset for each of those vertices. These values are listed in sequence, so we first ignore the vertices and focus on the x, y, z values. Essentially, these x, y, z values indicate which axes have been offset, so we can also ignore these values.

In total, up to 24 values are needed to represent which of the 24 axes (8 vertices × 3 axes) have been offset. However, storing offsets for all vertices, including those that have not been offset, would be very space-inefficient. Therefore, only the offsets for the moved axes are stored.

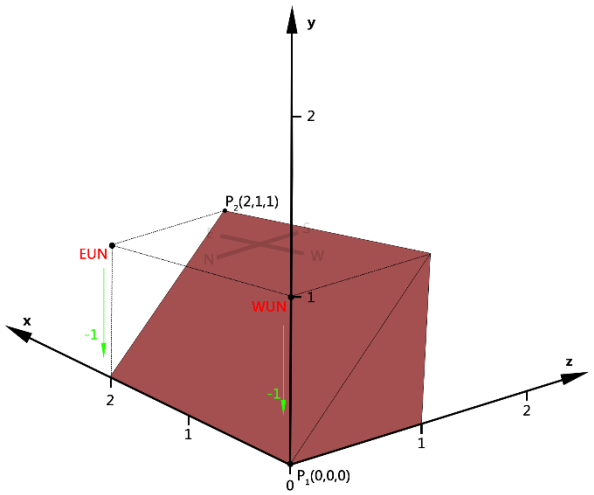
In our example, since we offset the Y values of EUN and EUS, we will need two values in the data to represent these offsets. The order is written from right to left, so we write the offset value for EUN's Y first, followed by EUS's Y. When reading the data, we check which axes have been offset and then read the corresponding values from the data in right-to-left order, assigning 0 to any axes that have not been offset.

These offset values are stored as 32-bit integers, but the offset amount itself only requires 16 bits. Therefore, when reading, we need to split the 32-bit signed integer into two 16-bit parts, each representing a separate offset value.

Thus, each number from the 8th value onward in the array actually represents two offset values.

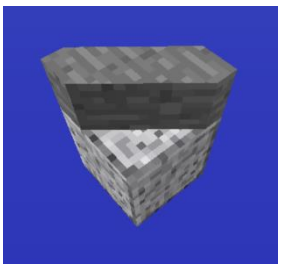
Another more straightforward example is:

```
Tiles info (BLOCK):
    block coord: 15 4 10
    grid: 2
    id: minecraft:littletilestileentity
    boxes count: 1
    Tile info:
        block id: minecraft:stained_hardened_clay:6
        boxes:  [0, 0, 0, 2, 1, 1, -2147475454, -1]
        Angle change state (from -2147475454):
        |FLIPPED|WDS|WDN|WUS|WUN|EDS|EDN|EUS|EUN|
        | EWSNUD|zyx|zyx|zyx|zyx|zyx|zyx|zyx|zyx|
        | 000000|000|000|000|010|000|000|000|010|
        Angle offset (from -1):
        WUN:  y_offset: -1
        EUN:  y_offset: -1
tips:
  P_1(0,0,0) 来自 [0,0,0,2,1,1,...]
  P_2(2,1,1) 来自 [0,0,0,2,1,1,...]
  -1 的二进制表示为 11111111,11111111,11111111,11111111 (32bit)
  WUN y_offset 来自前16位，即 11111111,11111111
  EUN y_offset 来自后16位，即 11111111,11111111
  grid 代表该方块内的小方块精度是 2
```

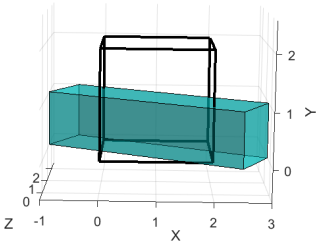


For some other slopes, when you calculate their offsets, they might exceed the boundaries of the block. For example, consider the small block on the right with a precision of 2. The offset values for each corner are as follows:

WDS:	x_offset: -2	z_offset: 2
WDN:	x_offset: 3	z_offset: -2
WUS:	x_offset: -2	z_offset: 2
WUN:	x_offset: 3	z_offset: -2
EDS:	x_offset: -3	z_offset: 2
EDN:	x_offset: 2	z_offset: -2
EUS:	x_offset: -3	z_offset: 2
EUN:	x_offset: 2	z_offset: -2



We can observe that all its corners exceed the boundaries of the block. The actual effect is as follows:



What we see in the game is just the shape after being clipped by the block boundaries: